Activity Transition Graph Generation: How Far Are We?

JIAKUN LIU, Harbin Institute of Technology, China
PEIXIN ZHANG*, Singapore Management University, Singapore
HAN HU, Monash University, Australia
YONGHUI LIU, Australian National University, Australia
WEI MINN, Singapore Management University, Singapore
FERDIAN THUNG, Singapore Management University, Singapore
SHAHAR MAOZ, Tel Aviv University, Israel
ERAN TOCH, Tel Aviv University, Israel
DEBIN GAO, Singapore Management University, Singapore
DAVID LO, Singapore Management University, Singapore

Android applications (i.e., apps) are indispensable nowadays and are getting bigger and bigger with an increasing number of functionalities. To understand how to access functionalities in an app, prior studies proposed tools to model the transitions between functionalities with the activity transition graph (ATG). ATG is an important data structure and has been used for various Android app analyses, including app design, understanding, and testing. However, there is no benchmarking work on ATG generation. It is still unclear whether the transitions identified by tools are correct and how many transitions are missed.

To fill this gap, we manually identified all transitions in 98 applications to build a benchmark. Using the benchmark, we evaluate seven popular ATG generation tools that were used in prior studies. We observe that these tools not only report incorrect transitions but also missed transitions, and different tools do not report the same set of transitions. Compared with the transitions reported by a single tool, the union set of the transitions reported by different tools contains fewer missed transitions but more incorrect transitions. We summarize five potential reasons that explain why GUI testing tools fail to identify transitions in ATG, revealing the limitations in the current design of exploration strategies. For instance, we observe that learning-based tools may overlook subtle distinctions between states, potentially misclassifying different states as identical. This will lead the tool to always focus on the old state while the new state is not fully explored. Based on our findings, we propose a series of suggestions for researchers using and building ATG generation tools. For example, when aiming to identify more transitions, one can combine the results of different tools by running each tool for 10 minutes, which will produce better results than running a single tool for 60 minutes.

CCS Concepts: • **Software and its engineering → Software testing and debugging**.

Additional Key Words and Phrases: Android, Activity Transition Graph, Program Analysis

* Corresponding author.
Authors' addresses: Jiakun Liu, Harbin Institute of Technology, Harbin, China, jiakunliu@hit.edu.cn; Peixin Zhang*, Singapore Management University, Singapore, Singapore, pxzhang@smu.edu.sg; Han Hu, Monash University, Melbourne, Victoria, Australia, agmaiofhuhan@gmail.com; Yonghui Liu, Australian National University, Australia, Yonghui.Liu@anu.edu.au; Wei Minn, Singapore Management University, Singapore, Singapore, wei.minn.2023@phdcs.smu.edu.sg; Ferdian Thung, Singapore Management University, Singapore, Singapore, ferdianthung@smu.edu.sg; Shahar Maoz, Tel Aviv University, Israel, Israel, maoz@cs.tau.ac.il; Eran Toch, Tel Aviv University, Israel, Israel, erant@tauex.tau.ac.il; Debin Gao, Singapore Management University, Singapore, Singapore, dbgao@smu.edu.sg; David Lo, Singapore Management University, Singapore, Singapore, davidlo@smu.edu.sg.

ACM 1049-331X/2025/1-ART1
https://doi.org/10.1145/3776553

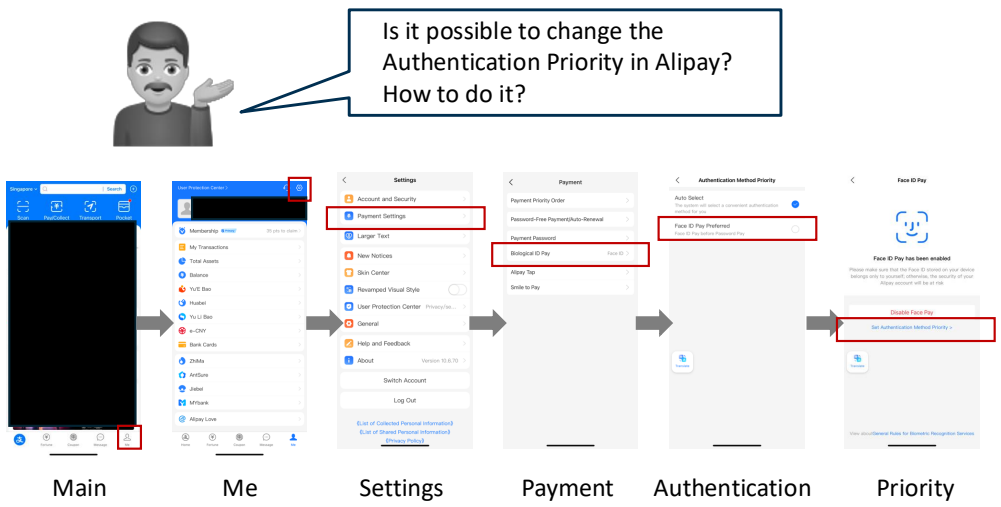


Fig. 1. An example of a functionality that is difficult to access. This example shows that it is difficult for a user to know how to access all functionalities in an app.

ACM Reference Format:
Jiakun Liu, Peixin Zhang*, Han Hu, Yonghui Liu, Wei Minn, Ferdian Thung, Shahar Maoz, Eran Toch, Debin Gao, and David Lo. 2025. Activity Transition Graph Generation: How Far Are We? . *ACM Trans. Softw. Eng. Methodol.* 1, 1, Article 1 (January 2025), 28 pages. https://doi.org/10.1145/3776553

1 INTRODUCTION

Android applications (i.e., apps) are indispensable in daily life nowadays. To attract more users, developers usually equip their apps with more features to satisfy the various usage scenarios of different users. For example, Alipay is a payment app that can be used to pay for goods and services, such as buying air tickets, booking hotels, ordering takeout, and calling taxis.

Understanding how to access all functionalities in an app is not easy. Figure 1 shows an example in which a user (or other stakeholders besides the developers, e.g., a researcher who analyzes the app) may not know whether there is a certain functionality in an app, e.g., setting up the priority of Authentication Method (face ID, fingerprint, password, etc.) in AliPay, and how to access it when exploring the app for the first time. To configure the Authentication Method Priority in Alipay, users should navigate to the Me section and tap the gear icon at the top right corner. This will take them to the Settings section, where they need to select the Payment Settings option. Clicking the Biological ID pay button will lead them to the Payment section, where they will find a button labeled “Set Authentication Method Priority”, which allows them to establish their preferred authentication method priority. Clearly, especially for medium to large apps, it is difficult to discover all the functionalities in an app without a comprehensive exploration.

To understand how to access all functionalities in an app, prior studies [6, 10, 12, 13, 24, 27, 33, 35, 39, 41] use the activity transition graph (shortened as ATG) of an app to model how to access all functionalities. An activity is a single, focused thing that the user can do,[1]. It provides the window in which the app draws its UI[2]. They consider an activity corresponding to functionality,

[1]https://developer.android.com/reference/android/app/Activity
[2]https://developer.android.com/guide/components/activities/intro-activities

and the transition from one activity to another indicates how to access the functionality. ATG is an important data structure. This is because ATG is used in various Android app analyses for app understanding and design [6, 10, 27], testing [12, 13, 17, 21, 24, 33, 35, 39, 41], etc. For example, if a transition is missed by a testing tool, then the functions related to that transition would be missed by the tool, leading to lower coverage of the testing tool and bringing potential threats to the quality of the apps.

Prior studies have proposed a series of tools to generate ATG. For example, Chen et al. [10] designed StoryDroid that extracts the transitions between activities statically and renders the UI of the initial state of each activity. Following that, Zhang et al. [40] proposed SceneDroid that improved the performance of StoryDroid. Specifically, SceneDroid smartly explores the apps with state fuzzing techniques. This strategy enables it to explore more scenarios, obtain more transitions between activities, and take a screenshot for each activity. Besides these studies, Liu et al. [25] designed an intelligent tool to predict the transitions between activities in apps automatically.

Moreover, some tools can generate ATG as a side effect or by-product. For example, one activity can be invoked from another using Inter-Component Communication (i.e., ICC). Therefore, the studies related to ICC resolution also reported the ATG generated by statically analyzing apps [37]. Users can also switch between activities by interacting with the UI of an app. Therefore, the transition from one activity to another can also be observed during GUI testing, where the GUI testing tools interact with the UI of apps [15, 16, 19, 26, 30, 31]. Existing ATG generation studies [9, 25, 40] commonly include these works as baseline approaches.

Unfortunately, when evaluating the performance of these tools, prior studies only compared the number of transitions obtained by their tools. It is still unclear whether the transitions identified by these tools are correct and how many transitions are missed by these tools. Clearly, low-quality ATGs will have detrimental impacts on various analyses that rely on them. This motivates us to explore: **How far are existing tools in generating high-quality activity transition graphs?**

To answer this question, we manually identify all transitions in 98 apps collected from F-Droid and AndroZoo [1, 4]. Then, we select seven state-of-the-art tools that are popularly used in ATG generation tasks, i.e., APE [16], FastBot2 [26], Stoat [31], Q-Testing [30], Humanoid [19], Monkey [15], and SceneDroid [40]. These tools are selected because they can generate ATG through GUI exploration that can provide the visual information of the activities (e.g., screenshots), which is helpful for users to understand the functionality of the app. The tools that can generate ATG through static analysis (e.g., ICCBot [37]) are not selected because they cannot provide the visual information of the activities and already have been evaluated in prior studies [36]. We first set up the environment for each tool, ensuring their correct execution. Then, we parse the results of each tool. With these data at hand, we answer the following research questions:

(1) **How effectively and efficiently can these tools generate the ATGs?**
We observe that all ATG generation tools report incorrect transitions and miss transitions. Different tools report different sets of transitions. Compared with the transitions reported by any single tool, the union set of the transitions reported by different tools contains more correct transitions and more incorrect transitions. Among the seven tools we investigated, APE is the best tool that reports the least incorrect transitions and misses the fewest transitions. APE is one of the fastest tools as well.

(2) **Why do ATG generation tools report incorrect transitions?**
We observe that some tools record the start of the activity rather than the real display of the activity, while some activities can be used for activity management, service management, background workers, etc., and never display on the screen. Besides, tools aggressively explore

the app before the UI page is fully rendered, leading them to record the corresponding activity of an explored widget incorrectly.

(3) **What challenges are faced by tools in finding transitions?** We identify five limitations of these tools in identifying transitions between activities. For example, we observe that vision-based tools, e.g., Humanoid [19], could incorrectly consider the same state as different states due to subtle visual differences. In contrast, learning-based tools, e.g., Q-Testing [30], may overlook subtle distinctions between states, potentially misclassifying different states as identical. The tools that consider the attributes of the widgets in a screen, e.g., Stoat [31], may fail to capture the semantic knowledge in textual information.

Based on our findings, we provide actionable suggestions to users and researchers. For example, when aiming to identify more transitions, users can combine the results of different tools with a 10-minute run for each tool, which will produce better results than running a single tool for 60 minutes. Researchers could combine different types of information (i.e., attributes of widgets, visual information, and semantic information of text) together to distinguish states and prioritize the widgets to explore.

In summary, the paper makes the following contributions:

(1) We manually identified all transitions in larger-scale apps collected from AndroZoo and F-Droid. We open-sourced the benchmarking dataset for future researchers.
(2) We empirically evaluate the performance of popular ATG generation tools. To the best of our knowledge, our work is the first work that focuses on the failures of such tools.
(3) Based on our findings, we proposed actionable suggestions for users and researchers.

The remainder of this paper is organized as follows: We first present how we collect related tools in Section 2. Then, we present how we implement the selected tools, how we collect the apps we experiment with, and how we manually identify all transitions in the collected apps in Section 3. We answer our research questions in Section 4. Specifically, we compare the results generated by different ATG generation tools with the ground truth ATGs in Section 4.1. We analyze the possible reasons for the reported incorrect transitions in Section 4.2, and in Section 4.3, we analyze the possible reasons for the missed transitions. We discuss the potential of improving the performance of ATG generation tools by combining the results of all tools with a short execution time in Section 5.1. Based on our findings, we also present the implications of the paper in Section 5.2 and discuss the threats to validity in Section 5.3. Finally, Section 6 concludes the paper and discusses potential future work.

2 RELATED WORK AND SELECTED TOOLS

In this section, we present existing studies on the generation of ATGs of Android apps and works that benchmark Android analysis tools.

2.1 Generating Activity Transition Graph

To better understand the progress of research related to generating ATGs, we first collect all papers that target it. To do so, we collect papers from ACM Digital Library, IEEE Explorer Digital Library, and Digital Bibliography & Library Project (i.e., DBLP) by searching the following keywords: “activity transition graph” and “android”. The search engine of these libraries could search the content of the paper. Therefore, if the paper mentions the “activity transition graph” and “android” at the same time, the search engine of these libraries would consider the paper to be related to the search query. Moreover, we use the advanced search function in the search engine and only

retrieve the papers published in top venues in the field of software engineering, i.e., ICSE, FSE, ASE, ISSTA, TOSEM, and TSE. As a result, we collect 30 papers.

Following that, we manually filter out the papers that do not focus on generating ATGs. Specifically, we manually read the collected papers to understand whether the paper targets generating ATGs. As a result, only three papers focus on generating ATGs for Android apps. The remaining papers either use the generated ATG results or mention the ATG usage scenario. Then, we read the selected three papers and involve more papers that are cited by these three papers as baseline approaches. These papers do not specifically generate ATG, but the tools proposed by these papers could also generate ATG as a by-product.

2.1.1 ATG Generation Tools. We first present the ATG generation tools analyzed in our study:

- Monkey [15], a pure **random** testing tool, testing Android applications by emitting pseudo-random streams of UI events (e.g., touch, gestures, random texts) and some system events (e.g., volume controls, navigation). Monkey is widely used in the industry for stress testing due to its ease of use and compatibility with any Android version; Monkey serves as a popular baseline for evaluating new testing techniques.
- Stoat [31] is a stochastic **model**-based testing tool. Given an app as input, Stoat uses dynamic analysis enhanced by a weighted UI exploration strategy and static analysis to reverse engineer a stochastic model of the app's GUI interactions. Then Stoat adapts Gibbs sampling to iteratively mutate/refine the stochastic model and guides test generation from the mutated models toward achieving high code and model coverage and exhibiting diverse sequences. During testing, system-level events are randomly injected to further enhance the testing effectiveness.
- APE [16] is a **model**-based GUI testing tool. Different from prior model-based testing tools like Stoat, which uses static GUI abstraction criteria, APE uses the runtime information to dynamically evolve its abstraction criterion via a decision tree, which can balance the size and precision of the model. Specifically, with this dynamically refined model, APE generates UI events via a random and greedy depth-first state exploration strategy. Moreover, APE also internally utilizes Monkey to occasionally emit random UI events and system events to avoid being stuck at local states.
- Humanoid [19] is a **deep learning**-based testing tool. Its core is a deep neural network model that predicts which UI elements on the current GUI page are more likely to be interacted with by users and how to interact with them. The model was trained on a large-scale crowd-sourced dataset of human interactions [11]. Humanoid is expected to accelerate GUI exploration toward important states faster because it prioritizes UI elements based on their importance and meaningfulness, just like a human.
- Q-Testing [30] is a **reinforcement learning**-based testing tool that employs a trained neural network to compare GUI pages. If a page resembles any of the previously explored GUI pages, the comparator provides a small reward. Conversely, if a page is dissimilar, it receives a substantial reward. These rewards are utilized and iteratively updated to guide the testing process, ensuring that it covers a broader range of functionalities within applications.
- FastBot2 [26], a **model**-based GUI testing tool, introduces a reusable automated technique for Android apps to accelerate testing. It leverages a probabilistic model to memorize and reuse execution results from previous testing runs. This approach involves a reinforcement learning algorithm-enhanced model-based guided testing strategy.
- SceneDroid [40] is a **static-analysis-guided ATG generation tool**. Specifically, SceneDroid designs an exhaustive exploration strategy to explore all scenes of an app and interact with as many interactive UI components as possible. SceneDroid also introduces state

fuzzing techniques to improve scene transition coverage. Most importantly, SceneDroid designs an indirect launch strategy that leverages already explored activities to launch activities that IC messages failed to launch indirectly.

Besides these tools, researchers also proposed other tools that target generating ATG. Specifically, Chen et al. [10] proposed StoryDroid, the first work that targeted generating ATG and obtained rendered UI at the same time. However, recently, they proposed a newer version of the StoryDroid, i.e., SceneDroid [40], which can improve the scene transition coverage by state fuzzing techniques. Therefore, we exclude StoryDroid [10] and use the newer version of the tool (SceneDroid [40]) in our paper.

Moreover, many works focused on generating ATG statically. For example, Liu et al. [25] and Liu et al. [20] used deep learning-based techniques to generate ATG. Octeau et al. [28, 29], Bosu et al. [7] and Yan et al. [37] focused on the ICC analysis. The by-product of their work can generate the ATG of apps statically. However, as they could not generate screenshots of activities, which makes it difficult for users to understand the functionality of the app, we excluded these tools from our paper.

Many tools used the static ATG to guide the model-based dynamic analysis. For example, Azim et al. [5] proposed A^3E to explore apps systematically. Lai and Rubin [18] proposed GoalExplorer for goal-driven exploration. As (1) the precision of the generated ATG is the same as that of the static tools, the difference between their approach and the static tools is that they can generate screenshots, and (2) both of them are out of date and are not well maintained[3]. We exclude them from our study.

Some prior studies also considered the transitions between windows (WTG) [20, 38] and screens (STG) [18]. They considered an activity, a menu, and a dialog as a window [38] and an activity that is decorated by fragments, menus, dialogs, etc., as a screen [18]. We focus on the transitions between activities because activity is the core component officially defined by Android and is managed by the system with activity stacks and manifest files, which is more like a developer-defined feature in an app. Moreover, "activity" is a more abstract granularity than "window" and "screen" (an activity can have more than one window and screen); ATG is a more abstract representation of other *TGs (two different windows in one activity triggering the same another activity is considered as two edges in WTG, but is considered as one edge in ATG). As the first paper benchmarking the performance of different *TG generation tools, we want to start by understanding the ability of different tools to generate ATG. If a tool cannot generate ATG well, then the tool may not be able to generate finer-grained *TG well. We suggest that future work can focus on the generation of finer-grained *TG.

2.2 Benchmarking Android Analysis Tools

With the advancement of Android-related analysis research, creating competing tools for a task, many researchers benchmarked the performance of these tools. For example, Akinotcho et al. [3] benchmarked the performance of automated testing tools for Android apps in terms of activity coverage, and manually identified the reasons for missing activities. They observed that the tools could only cover 30% of activities in the app, and the main reason for missing activities is the missing necessary conditions to launch the activities. Different from their work, our paper addresses not only whether an activity is covered but also whether all transitions that can access it are covered, which is more challenging than simply visiting the activities. Moreover, Wang et al. [34] benchmarked the

[3]The link to repository provided by Azim et al. [5] for A^3E is not valid when we wrote the paper: http://spruce.cs.ucr.edu/A3E/. As indicated in https://github.com/resess/GoalExplorer, GoalExplorer [18] requires API levels 25 and under, and the apps under test should also not use *androidx* libraries. This makes it useless with recent apps.

test generation tools for Android apps. They observed that Monkey achieved the highest method coverage on 22 of 41 apps, and Stoat could trigger the highest number of unique crashes on 23 apps. Su et al. [32] benchmarked the ability of tools to find crash bugs. They constructed 52 real and reproducible crash bugs and identified reasons that block tools from finding bugs, e.g., specific user interaction patterns. Different from their work, our paper focuses on benchmarking the transitions between activities, which has potential for bug detection and is useful for understanding the functionality of the app. This task requires the tool to explore all possible transitions to all activities, rather than simply visiting the activities. Therefore, our work is different from prior studies.

The most related work to our paper is the work by Yan et al. [36], which benchmarked the performance of different static tools (e.g., A^3E [5], IC3 [28], GATOR [38], ICCBot [37]) in terms of ICC resolution. These tools generate ATG as a by-product through the static analysis of the app, rather than the dynamic exploration of the app. They observed that up to 38%-85% ICCs are missed by tools, and wrongly reported ICCs are mainly sent from or received by only a few components. Different from their work, our paper focuses on benchmarking the tools that can dynamically explore the transitions between activities (e.g., APE and Stoat, that are not explored in Yan et al. [36]'s work). The dynamic exploration of the app is more challenging than the static analysis of the app, as it requires the tool to interact with the app and explore all possible transitions. At the same time, the dynamic exploration of the app can also generate screenshots of activities, which is helpful for users to understand the functionality of the app. To achieve this goal, we manually identify all transitions in collected apps as the ground truth transitions. Then, we compare the results of different tools with the ground truth transitions to understand the possible reasons for missing transitions and the incorrect transitions reported by the tools. Therefore, our work is different from Yan et al. [36]'s work.

3 APPROACH

3.1 Implementing the Selected Tools

To implement the selected tools, we re-use the infrastructure provided by Su et al. [32], where they refactored the implementation of different tools and provided a command line to execute the tools. However, some of the tools studied in our work are not covered by Su et al. [32]'s work, and they did not provide readily used execution environments for different tools. It is difficult to set up the execution environment of different tools. For example, to run Stoat, users have to use Ruby 2.1 and the corresponding dependency Nokogiri[4], and their installation requires many third-party libraries. This motivates us to set up a docker image for each tool.[5]

Moreover, to obtain the ATG from different tools, we need to parse their results, especially for the tools that do not specifically target the generation of ATGs. We noticed that the Android system recorded the start and the display of activity with the help of ActivityManager in the Logcat file. However, we can not simply parse the ATG from the Logcat file. This is because the tools can explore the app for multiple rounds with multiple exploration sessions, and the Logcat does not record the information related to the border of different sessions. Therefore, the ATG recorded in the Logcat is not the real ATG. Though prior studies used tools that do not target generating ATG as baseline approaches, they did not present how they extracted the ATG from the output of the tools. This indicates that we need to carefully understand the output of different tools.

We set up the different tools as follows:

[4]http://www.nokogiri.org/tutorials/installing_nokogiri.html

[5]Replication package: https://github.com/jiakun-liu/ATGBenchmarking

- Monkey. We set up Monkey using the default settings provided by Google.[6] When Monkey starts a new session, it will log "`:Switch:`" at the beginning of a line. If the exploration starts a new activity, Monkey will log "`// Allowing start of Intent {`" at the beginning of the line. We extract all the explored activities, and consider there is a transition between two consecutive explored activities within a session. However, Monkey does not take the screenshots of an activity. To capture the screenshots of the explored activity, we set up another thread that keeps taking screenshots. We do not set a time interval between taking screenshots. The time interval depends on the time to execute the command.
- APE. We use the implementation of APE from Su et al.'s [32] work. When APE starts a new transition, it will log "`[APE] === Adding edge...`", and indicate the source and target of the transition using "`[APE] Source:`" and "`[APE] Target:`". APE takes the screenshots of the activities automatically, and the screenshots are saved in the SD card of the emulator. We pull the screenshots after running APE.
- Stoat. The main obstacle when we set up Stoat is the environment in which the tool is run. To overcome this obstacle, we manage the Ruby environment using rbenv[7] and install Ruby from source using ruby-build[8]. We also install other dependencies on the Linux-based docker image. Stoat generates the states (a state is determined by the activity and its corresponding widgets) of the apps in `allstates.txt` and the transitions between states in `app.gv`. We parse the results in these files to obtain the final ATG. Stoat captures the screenshots as we all the layout of each app state and save them in the `ui` directory.
- FastBot2. We set up FastBot2 using the default settings provided by ByteDance.[9] When FastBot2 starts a new session, it will log "`not testing app, need inject restart app`". If the exploration starts a new activity, FastBot2 will log "`// current activity is `". We extract all the explored activities, and consider there is a transition between two consecutive explored activities within a session. FastBot2 takes the screenshots of the activities automatically, and the screenshots are saved in the SD card of the emulator. We pull the screenshots after running FastBot2.
- Humanoid. We use the implementation of Humanoid from Su et al.'s [32] work. Humanoid generates the states of the apps and the transitions between states in `utg.js`. We parse the results in these files to obtain the final ATG. Humanoid captures the screenshots as we all the layout of each app state and save them in the `views` directory.
- Q-Testing. We use the implementation of Q-Testing from Su et al.'s [32] work. When Q-Testing starts a new session, it will log "`the chosen action is restarting app`". If the exploration starts a new activity, Q-Testing will log "`the jumped state is`". We extract all the explored activities, and consider there is a transition between two consecutive explored activities within a session. However, Q-Testing does not take a screenshot of an activity. To capture the screenshots of the explored activity, we use the same approach when we use Monkey.
- SceneDroid. It took us two weeks to implement SceneDroid. We discussed with Zhang et al. [40] and updated the code of SceneDroid. For example, we find that SceneDroid needs a specific version of apktool.[10] SceneDroid records the transition results in `iccbot.txt` and screenshots in `screenshot` directory. However, we notice that `iccbot.txt` also contains the ATG generated by static ATG generation tool ICCBot. Therefore, we parse the log of

[6] https://developer.android.com/studio/test/other-testing-tools/monkey
[7] https://github.com/rbenv/rbenv
[8] https://github.com/rbenv/ruby-build
[9] https://github.com/bytedance/Fastbot_Android
[10] https://apktool.org/blog/apktool-2.7.0/

SceneDroid that is saved in `scenedroid.log`. Specifically, when SceneDroid explores a new activity, it will record the current activity with "`[screen.act] :`" at the beginning of the line, and record the newly explored activity with "`[NEW ACT] :`" at the beginning of the line. When SceneDroid explores a transition that is not identified in the static analysis process, it will record the transition with "`[NEW Trans] :`" at the beginning of the line. We parse the log of execution of SceneDroid to get the final ATG.

The Docker image contains a Google Android 9 emulator (API level 28) with a 1GB SD Card and an X86 ABI image (with other configurations using the default settings of AVD Manager[11]). This is because this system images (API level 28 with X86 ABI image) support ARM by default and provide dramatically improved performance when compared to those with full ARM emulation.[12] By doing so, we can explore the tools in parallel without considering the limitations of emulators.

We conduct our experiment on a 64-bit Ubuntu 22.04.5 LTS machine (128 cores, AMD EPYC 7763 CPU, and 256GB RAM). Following prior studies [32], for each app under test, we run each tool for one hour. Considering the flaky nature of the dynamic analysis, following a prior study that focused on benchmarking Android test generation tools [34], for each tool, we run it on each app three times and select the result with the most transitions as the result of the tool on this app. We can only run up to 15 docker containers at the same time due to the limitation of Android [32].

3.2 Collecting the Apps to Test

We download the app index from F-Droid[13] and AndroZoo [4]. F-Droid is the largest platform that hosts open-sourced apps, while AndroZoo is the largest platform that hosts more than 24 million apps sourced from various markets (including Google Play, PlayDrone, AppChina, etc). Then, we select apps with the following requirements:

(1) The apps have been released recently and the minimal support SDK version should be lower than 28. Prior studies only tested apps that were released several years ago. However, with the development of Android, many new features are proposed by Android. To understand the performance of the selected tools on the recent apps, we collect the apps that were released after October 2022 (2 years before we started our work). In addition, we only consider apps that support Android SDK version lower than 28, as some tools we evaluate in this paper are not compatible with Android SDK version 28 and above.
(2) The apps have more than two activities declared in the manifest file.[14] To understand the transitions between activities, the apps under test should have enough transitions to be tested.
(3) The apps with native code should use a common Application Binary Interface (ABI) that can be run on an emulator hosted on an x86_64 Linux server. This is because we conducted our experiment using an x86_64 Linux server that can only host an Android emulator with an x86 and x86_64 CPU instruction set. If the app does not support x86 or x86_64, i.e., only supports running on armeabi-v7a and arm64-v8a, it cannot be included in our dataset.

Note that for the apps collected from AndroZoo and F-Droid, we exclude those with extensive internet access. This is because we do not want to test or analyze apps that are not allowed to be tested or analyzed, as this may be considered an attack on their servers. To avoid the potential bias

[11] https://developer.android.com/tools/avdmanager
[12] https://developer.android.com/studio/releases/emulator#support_for_arm_binaries_on_android_9_and_11_system_images
[13] https://f-droid.org/
[14] https://developer.android.com/guide/topics/manifest/activity-element

brought by this, we collect an additional 100 apps from HackerOne[15]. HackerOne is a platform that connects organizations with security researchers to find and fix vulnerabilities in their systems. Many companies use HackerOne to run their bug bounty programs, where they reward security researchers for finding and reporting vulnerabilities in their systems. The apps in the HackerOne bounty program are allowed to be tested and analyzed by developers. Specifically, we filter out the apps that do not satisfy the above requirements. Then we manually explore the apps and create an account for the ones that require login. However, during this process, we noticed that many apps require two-step verification (2SV). If the app requires 2SV, we exclude it from our dataset because it is challenging to automate testing.

We randomly collect 140 Android applications after filtering against the above criteria. We unzipped the collected APK files to see the architecture that the binary files can support. We also parse the manifest file to extract the minimal support SDK version and declared activities. The collected apps have 6 activities on average, and 14.48 activities on average, with a maximum of 137 activities. This indicates that the collected apps have rich transitions for tools to explore.

3.3 Manually Identifying the Transitions between Activities

After obtaining the ATG extracted from the results of the tools, we would like to understand the performance of each tool. Therefore, we need to identify all real transitions in the collected apps. To do so, we first collect an initial draft of transitions of the collected apps through manual exploration. Then, we refine and revise the draft transitions to obtain the real transitions of the collected apps using the results of the ATG generation tools as a supplementary reference. The detailed processes are as follows:

The first two authors of the paper individually and manually explored the apps to collect draft transitions at first. They recorded the explored transitions and took screenshots of the visited activities to obtain an initial draft of transitions.[16,17,18] Note that as we focus on benchmarking the tools that can generate ATG through GUI exploration, following prior studies [26, 31], we do not consider the activities that are not visible to users.

Then, **the first two authors of the paper refined and revised the initial draft of transitions using the results of the ATG generation tools as a supplementary reference to obtain the real transitions.** To do so, the first two authors of the paper compared the draft transitions and the transitions reported by ATG generation tools. Specifically,

- If a transition was reported in the ATG generated by tools rather than the draft transitions, the authors would explore the app again to see whether there should be such a transition. The authors also referred to the execution log of the tool to understand how the tool explored the transitions.
- If a transition was observed in the draft transitions rather than in the result of a specific ATG generation tool, the author would refer to the execution log of the tool, the recorded screenshots, the source code provided by the paper, and the original paper to understand the potential reason. The author also looked into the code for open-sourced apps.

A three-iteration open-coding process was conducted to understand the differences between the draft transitions and the ATG reported by different tools. During this process, the draft transitions are also revised and refined to obtain the real transitions. More specifically, the first author of the

[15]https://hackerone.com/directory/programs?asset_type=GOOGLE_PLAY_APP_ID&order_direction=DESC&order_field=launched_at

[16]https://stackoverflow.com/questions/13193592/getting-the-name-of-the-current-activity-via-adb

[17]https://stackoverflow.com/questions/27766712/using-adb-to-capture-the-screen

[18]https://stackoverflow.com/questions/26586685/is-there-a-way-to-get-current-activitys-layout-and-views-via-adb

paper drafted a coding schema of the potential reasons for the mismatch using the first 20 apps. The second author of the paper then used the coding schema to label the reasons for the same set of 20 apps. Specifically, we recorded the class (which are generally a subclass of View[19], e.g., Switch[20], TextView[21]) of the widget that needed to be interacted to trigger the transition, and the potential reasons for the actions not being triggered by the tool. If a transition to be triggered requires multiple actions, we record the actions in the order they need to be performed. For example, for transition from `main.TimerActivity` to `statistics.main.StatisticsActivity` (i.e., `main.TimerActivity` → `statistics.main.StatisticsActivity`) in `com.apps.adrcotfas.goodtime`, we also record the action needed to trigger the transition, i.e., click a `ImageButton` then a `widget.CheckedTextView` button (i.e., `ImageButton` → `widget.CheckedTextView`). If a transition needs to be triggered from a certain activity, while this activity is not visited, we consider it as a missing pre-step. If a transition needs to be triggered with more than three actions on the current activity, we consider it as a complex logic. A discussion is held to resolve disagreement, and the coding scheme is refined and revised. Finally, the first two authors of the paper manually labeled the rest of the apps to obtain the real transitions of the collected apps. Section 4.2 presents the coding schema of the reasons for the wrongly reported transitions, and Section 4.3 presents the coding schema of the reasons for the transitions missed by the tools.

During this process, we observed that some apps only have one transition. We exclude the apps that only have one transition in our final dataset. Finally, 98 apps are collected in the benchmark dataset. 43 apps are collected from Google Play via AndroZoo, 10 apps are collected from the HackerOne index and downloaded from Google Play, and 45 apps are collected from F-Droid. The average number of transitions of the collected apps is 11.3, and the median is 6.

3.4 Quantitative Analysis

To understand the effectiveness of each tool, given the ground truth transitions identified by humans in Section 3.3, we would like to evaluate the performance of each tool using precision, recall, and F1-scores. Specifically, the precision of a tool in identifying ATG transitions is calculated as the proportion of the real transitions among the transitions reported by the tool, i.e.,

$$Precision = \frac{\#(real\ transitions \cap reported\ transitions)}{\#reported\ transitions}$$

The recall of a tool in identifying ATG transitions is calculated as the proportion of real transitions reported by the tool among all the real transitions identified by humans, i.e.,

$$Recall = \frac{\#(real\ transitions \cap reported\ transitions)}{\#real\ transitions}$$

The F1-score of the tool in identifying ATG transitions is calculated as the harmony means of precision and recall that symmetrically represents both precision and recall in one metric. i.e.,

$$F1-score = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

To understand the efficiency of different tools in identifying transitions between activities, we would like to investigate the time to visit an activity and the time to explore a transition. **An activity can be accessed through different transitions. An activity being accessed does not mean that all related transitions have been explored.** To do so, we parse the Logcat generated by the Android system. Specifically, Logcat records the time stamp when an activity is `Displayed`.

[19]https://developer.android.com/reference/android/view/View
[20]https://developer.android.com/reference/android/widget/Switch
[21]https://developer.android.com/reference/android/widget/TextView

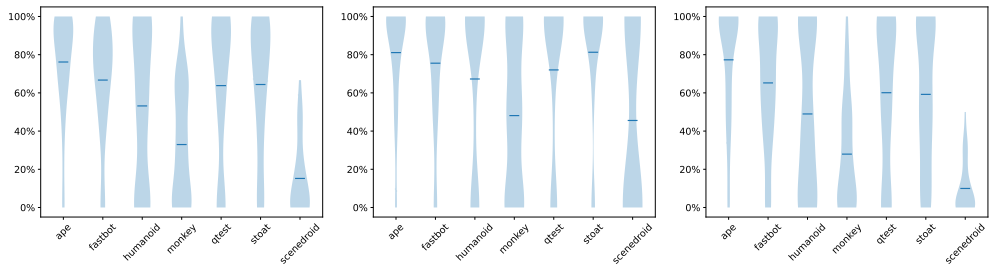
Since an activity can be visited and a transition can be explored multiple times when running the tool, we consider the time interval between the start of the tool and the first time an activity is displayed as the time to visit this activity; similarly, we consider the time interval between the start of the tool and the first time a transition is explored as the time to explore this transition. Note that Stoat automatically kills and restarts Logcat multiple times during the execution of the tool, which means there is no full Logcat for Stoat. Therefore, we exclude Stoat when discussing efficiency.

4 RESULTS

In this section, we would like to understand the performance of different tools in terms of generating ATGs. In Section 4.1, we first present the quality of ATG generated by different tools and compare them with the ground truth ATGs. We also explore the efficiency of obtaining ATGs using different tools. In Section 4.2 and Section 4.3, we analyze the potential reasons for the mismatch between the ATGs generated by tools and the ground truth ATGs.

4.1 RQ1: How Effectively and Efficiently Can These Tools Generate the ATGs?

4.1.1 Effectiveness. Table 1 shows the performance of different tools in terms of identifying all transitions in all collected apps. Figure 2 presents the distributions of precisions, recalls, and F1-scores of different ATG generation tools on each collected app. Figure 3 shows the common transitions and distinct transitions identified by different tools.



(a) F1-scores (b) Precisions (c) Recalls

Fig. 2. Performance of each tool across apps. The bar indicates the mean value. These figures show that these tools cannot recognize all transitions and may falsely report incorrect transitions.

Table 1. Overall performances of different tools. The darker the color, the better the result.

Tool	Precision	Recall	F1
APE	0.74	0.64	0.68
FastBot2	0.71	0.56	0.63
Humanoid	0.84	0.34	0.49
Monkey	0.55	0.19	0.28
Q-Testing	0.87	0.50	0.64
SceneDroid	0.63	0.03	0.04
Stoat	0.85	0.47	0.61

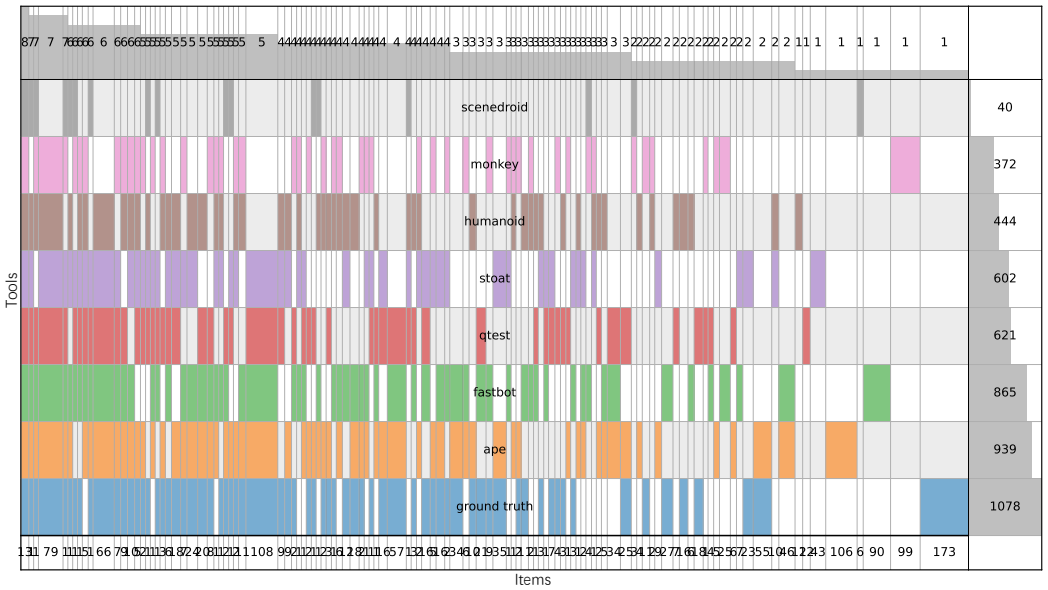


Fig. 3. Comparison between different tools. The colored cells show the transitions identified by each tool. Each row corresponds to a tool, and the tool name is shown in the middle of each row. Each column is a set of transitions recognized by k tools (highlighted in the colors), where the top row shows the 'k' and the bottom row shows the size of the transition set. The first column is the set recognized by all tools, whereas the last one is the set of 173 transitions recognized by no tools, thus only present in the ground truth. The rightmost column shows the total number of transitions reported by each tool (For example, there are 1,078 transitions in ground truth, and APE identifies 939 transitions). Note that transitions wrongly reported by the tools are included (i.e., the transitions that cannot be observed by humans, thus, the last row is not fully blue), and we will discuss this in Section 4.2. This figure shows that only (173 out of 1,078) 16.0% of transitions cannot be recognized by any tool. We suggest that we could use different tools at the same time to complement each other.

APE outperforms all other tools significantly (p-values < 0.05 and $|\text{Cliff's } \Delta| > 0.11$ in terms of F1-scores). Specifically, regarding the number of transitions, there are significant differences between the ATG identified by humans and the ATG reported by tools except for APE (p-values < 0.05 and $|\text{Cliff's } \Delta| > 0.11$ in terms of F1-scores). The number of transitions identified by APE is only slightly lower than that reported by humans, showing the promising results of APE. Though there are no significant differences between APE and other tools (except Monkey and SceneDroid) in terms of precision, there are significant differences between APE and all other tools in terms of recall (p-values < 0.05 and $|\text{Cliff's } \Delta| > 0.11$). This indicates that **ATG generation tools commonly can falsely report incorrect transitions, while APE can identify more transitions than other tools.**

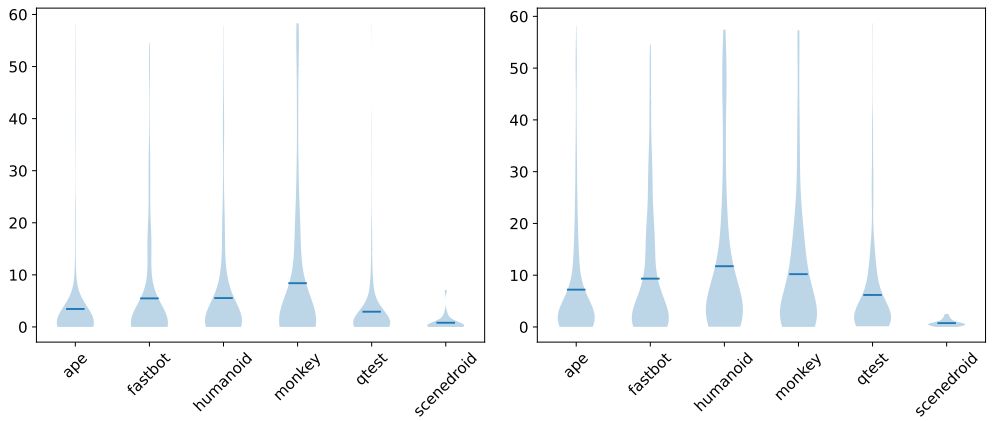
In contrast, **SceneDroid performs the worst** (p-values < 0.05 and $|\text{Cliff's } \Delta| > 0.11$ in terms of F1-scores) among all ATG generation tools. We carefully read the paper and the code, tracking the execution of SceneDroid step by step with 20 apps for over 2 weeks to ensure the correct execution of SceneDroid. However, we still observe that SceneDroid is the worst tool in generating ATG. This motivates us to inspect the potential reasons in Section 4.3. Following SceneDroid, Monkey is the second worst tool in identifying transitions in ATG in terms of recall. Considering Monkey is a random-based tool (as is shown in Section 2), this shows the limitation of the random-based tools.

There are no significant performance differences between FastBot2, Stoat, Q-Testing, and Humanoid. However, as indicated in Figure 3, these tools do not identify the same set of transitions. One possible reason is that these tools use different techniques, e.g., deep-learning-based techniques and model-based techniques, to explore the apps. This indicates that different tools have different advantages.

A potential solution to improve the performance of tools to identify ATG transitions is to combine the results from multiple tools. As a result, the F1-score of combined ATG is 0.70, which is higher than the best ATG generation tool (i.e., APE) by 2.1%. the recall of combined ATG is 0.83, which is higher than the best ATG generation tool (i.e., APE) by 30.1%. However, the precision of the combined ATG is 0.67, which is lower than the best ATG generation tool (i.e., Q-Testing) by 30.7%. This indicates that **simply combining the results of different ATG generation tools can identify more transitions but also can introduce incorrect transitions.** We also considered different combinations of the results of selecting only a few tools. We found that the combination of Humanoid, Stoat, and Q-Testing achieves the best performance in terms of F1-score (0.75), which is 6.5% better than combining the results from all tools. We will discuss the possible other strategies in combining the results of different tools in Section 5.1.

> In terms of the ability to identify transitions, APE is the best while SceneDroid is the worst. Besides, there are no significant performance differences between FastBot2, Stoat, Q-Testing, and Humanoid. However, these tools do not identify the same set of transitions. Simply combining the results of different tools can identify more transitions but can also introduce incorrect transitions.

4.1.2 Efficiency. Figure 4 shows the performance of different ATG generation tools regarding the time to visit activities and the time to explore transitions.



(a) The time for each tool to visit each activity (b) The time for each tool to explore each transition

Fig. 4. Performance of different tools in terms of time efficiency.

SceneDroid is the fastest tool to visit activities and explore transitions. One reason is that SceneDroid fails on the large apps, which leads to SceneDroid only visiting a limited set of activities and exploring a limited set of transitions, and it takes less time to explore these limited sets of activities and transitions because the explorations are usually taken at the beginning of

running these tools. Moreover, as indicated in Figure 4b, for SceneDroid, exploring transitions takes less time than visiting activities. This is because SceneDroid directly visits activities using the command line, and though there are new activities to be visited later, there are no new transitions to be explored.

APE, Q-Testing, and FastBot2 significantly faster then Monkey in terms of visiting activities. Though APE is the fastest tool in terms of visiting activities, there are no significant differences between APE, Q-Testing, and FastBot2. Among them, Q-Testing and FastBot2 are reinforce-learning-based dynamic testing tools, indicating the effectiveness of reinforce-learning in identifying unexplored activities. Monkey performs the worst in terms of visiting activities, indicating the drawbacks of the random-based tools.

However, **APE is significantly faster than FastBot2 in terms of exploring transitions. Q-Testing has a similar speed compared with APE while FastBot2 is almost as slow as Monkey.** The difference between the time to visit activities and the time to explore transitions illustrates the difference between these two metrics. Though many activities are explored, a huge number of transitions are explored later or even unexplored. Therefore, it is important to understand the performance of different tools in exploring transitions.

Humanoid costs almost the same time as Monkey in terms of visiting activities and is the slowest tool to explore transitions. One possible reason is that Humanoid uses how humans interact with apps to train their deep learning model, where humans may not target to explore all activities quickly.

> APE, Q-Testing, and FastBot2 are faster than Monkey when exploring activities. However, regarding the time to explore transitions, APE is the fastest while Humanoid is the slowest, and FastBot2 has a similar speed compared with Monkey. This shows that the ability to visit activities cannot represent the ability to explore transitions, indicating the necessity of benchmarking tools in terms of generating ATG.

4.2 RQ 2: Why Do ATG Generation Tools Report Incorrect Transitions?

In Section 3.3, we compared the results reported by the ATG generation tools and the draft transitions identified by humans. We find that the transitions reported by ATG generation tools may contain transitions we did not notice during our manual construction of the ATG benchmark. After manual verification, we observe many transitions are wrongly reported, which means they are impossible to achieve during manual exploration. Specifically, the selected tools wrongly report 60% of the apps among the manual analyzed 20 apps. We manually analyze the potential reason in Section 3.3, and we would like to present the results of how these tools wrongly report these impossible transitions here.

The tool wrongly considers the activity usage beyond GUI composition. For example, FastBot2 records the start of the activity. However, some activities are used as a router rather than a UI page that allows users to interact with it. Figure 5 shows an example. In `org.secure.privacy-friendlynotes_18`, the `SplashActivity` is used to redirect users to different activities based on whether it is the first time a user uses the app. If it is the first time for users to use the app, the `TutorialActivity` will be displayed; otherwise, the `Main` activity will be displayed. The `SplashActivity` will never be displayed. Therefore, a user can not visit this activity. In fact, an activity also can be used for activity management, service management, background workers, etc. We observe that 58.3% of apps are wrongly reported because the tools record the start of an activity rather than the display of the activity. This indicates that apps commonly use some activity beyond GUI composition. When ATG is used, e.g., to analyze code reachability, such transitions identified

```kotlin
class SplashActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        val mainIntent = if (FirstLaunchManager(this).isFirstTimeLaunch) {
            Intent(this, TutorialActivity::class.java)
        } else {
            Intent(this, MainActivity::class.java)
        }

        startActivity(mainIntent)
        finish()
    }
}
```

(a) SplashActivity in org.secure.privacyfriendlynotes

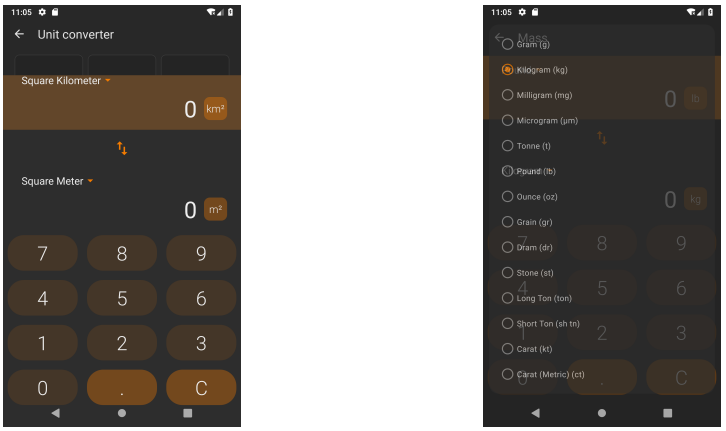
```
1  // the top activity is org.secuso.privacyfriendlynotes.ui.SplashActivity, phone
       capture activity is com.android.camera.CaptureActivity
2      // Allowing start of Intent { act=android.intent.action.MAIN cat=[android.
           intent.category.LAUNCHER] cmp=org.secuso.privacyfriendlynotes/.ui.
           SplashActivity } in package org.secuso.privacyfriendlynotes
3  // current activity is org.secuso.privacyfriendlynotes.ui.SplashActivity
4  Sleeping for 2000 milliseconds
5  // the top activity is org.secuso.privacyfriendlynotes.ui.TutorialActivity,
       phone capture activity is com.android.camera.CaptureActivity
6      // Allowing start of Intent { cmp=org.secuso.privacyfriendlynotes/.ui.
           TutorialActivity } in package org.secuso.privacyfriendlynotes
7      // activityResuming(org.secuso.privacyfriendlynotes)
8  // current activity is org.secuso.privacyfriendlynotes.ui.TutorialActivity
```

(b) A piece of fastbot.log that is related to SplashActivity

Fig. 5. An example of activity and its exploration log by FastBot2. This example shows that the tool records the start of the activity rather than the display of the activity.

could be useful. However, when we aim to explore the functionalities of an app and model the app from the frontend UI, we do not need to record such transitions.

The tool aggressively explores the app before the UI page is fully rendered. Figure 6 shows the screenshots of the app when the action to explore the app is taken when the activity is not fully rendered: the widgets of the last activity are still on the page, the widgets of the new activity appear. At this time, the system may detect that the current activity is still the last activity, but the widgets executed are in the new activity; or it may detect that the current activity is the new activity, but the widgets executed are in the last activity. In either case, it may cause incorrect transitions to be reported. We observe that 58.0% of the wrongly reported transitions are related to aggressive app exploration before the UI pages are fully rendered. We suggest that future researchers should wait for the full rendering of the UI pages to test.



(a) The 5th step for APE to explore the `UnitConverterPickerActivity` in `com.simplemobiletools.calculator`.

(b) The 13rd step for APE to explore the `UnitConverterActivity` in `com.simplemobiletools.calculator`

Fig. 6. Screenshots of the activities that are not fully rendered.

> ATG generation tools report incorrect transitions because (1) the tool wrongly considers the activity usage beyond GUI composition, and (2) the tool aggressively explores the app before the UI page is fully rendered.

4.3 RQ3: What Challenges Are Faced by Tools in Finding Transitions?

We manually analyze the potential reasons why transitions are missed by tools in Section 3.3. Here, we would like to present the results.

4.3.1 Why Do Tools That Use Static Analysis Fail? We first would like to understand the failure of the ATG generation tool that involves static analysis as a significant stage, i.e., SceneDroid.

Unlike other tools that solely rely on dynamic testing, SceneDroid uses the static analysis results as the input of the dynamic testing. Specifically, SceneDroid uses a specific version of `apktool` to unpackage and repackage the app to update the manifest file so that all activities can be invoked from the external command line. Then, SceneDroid conducts a static analysis to obtain the args used to launch the activity from the external command line.

However, we notice that this process is very sensitive to the app's API version and can always fail on certain APIs. Figure 7 shows examples of failures related to the static part of SceneDroid. We notice that the static part of SceneDroid can fail because of the crash of `Soot`, the tool used to perform static analysis. Moreover, the parsing process requires a long time and can be time out frequently. Besides, SceneDroid cannot handle the apps that are packed.[22] Furthermore, these static analysis tools cannot effectively analyze cross-platform applications developed in languages other than Java or Kotlin [8]. For example, apps built with React Native are particularly challenging because they compile into the unexplored bytecode format, Hermes Bytecode, that existing frameworks like Soot and APKtool cannot interpret [23]. Since these cross-platform apps don't use Android's traditional activity-based GUI management [22], current activity identification strategies have become ineffective. This would make SceneDroid fail to extract static information

[22]https://github.com/MagiCiAn1/APKProtectionSearch

```
soot.jimple.toolkits.typing.integer.InternalTypingException: Unexpected type
    [0..1] (Integer1Type)
        at soot.jimple.toolkits.typing.integer.ClassHierarchy.typeNode(
            ClassHierarchy.java:155)
        at soot.jimple.toolkits.typing.integer.TypeResolver.typeVariable(
            TypeResolver.java:121)
        ...
        at soot.PackManager$1.run(PackManager.java:1279)
        at java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor
            .java:1149)
        at java.util.concurrent.ThreadPoolExecutor$Worker.run(
            ThreadPoolExecutor.java:624)
        at java.lang.Thread.run(Thread.java:748)
The analysis is stopped, caused by Unexpected type [0..1] (Integer1Type)
```

(a) Stack traces of the crashed static analysis part in SceneDroid. The crash is caused by a certain version of the static analysis tool (Soot) that cannot be applied to a certain version of Android SDK.

```
============  NEW START ACTIVITY ============
[START ACTIVITY]:  at.jclehner.rxdroid.DoseHistoryActivity
[component]:  at.jclehner.rxdroid/at.jclehner.rxdroid.DoseHistoryActivity
[-] DON'T GET EXTRAS
[cmd]:  timeout 20s adb -s emulator-5554 shell am start -S -n at.jclehner.
    rxdroid/at.jclehner.rxdroid.DoseHistoryActivity -W
[-] Crash Start Activity:  at.jclehner.rxdroid.DoseHistoryActivity
[OTHER]:
[['android.intent.action.EDIT', 'android.intent.category.DEFAULT'], ['zxy', '
    zxy']]
```

(b) Logs related to failure to invoke an activity externally with the command line because of lacking necessary information.

Fig. 7. Examples of the failures related to SceneDroid.

and generate nothing to invoke activity externally, which leads to SceneDroid only being able to explore a limited set of activities.

4.3.2 Why Do Dynamic Tools Fail? Figure 8 shows the steps for tools to explore a new activity. Basically, the tool needs to determine whether the current state of the app is a new state. If it is a state that has been observed before, the tool considers this as an explored state, backtracks to the last state, and explores other exploitable components. If it is a new state, the tools will identify all interactive components on the app's UI. Then, the tool will prioritize the components to explore based on their implemented algorithms. Finally, the tool will interact with the UI with various actions, i.e., click the button, generate the input for the textbox, etc. After doing so, the app will be redirected to another state.

If the tool fails in any of the steps in this figure, the exploration of the ATG generation tool will fall into a loop, i.e., it will be trapped in the current activity without exploring more. In this subsection, we would like to understand the ability of different tools in the above-mentioned steps and characterize the UI of the activities that contribute to their failures.

Table 2 shows the distribution of the expected actions to explore the missed transitions. We observe that almost half of the missed transitions occur because a key step in exploring the app is not

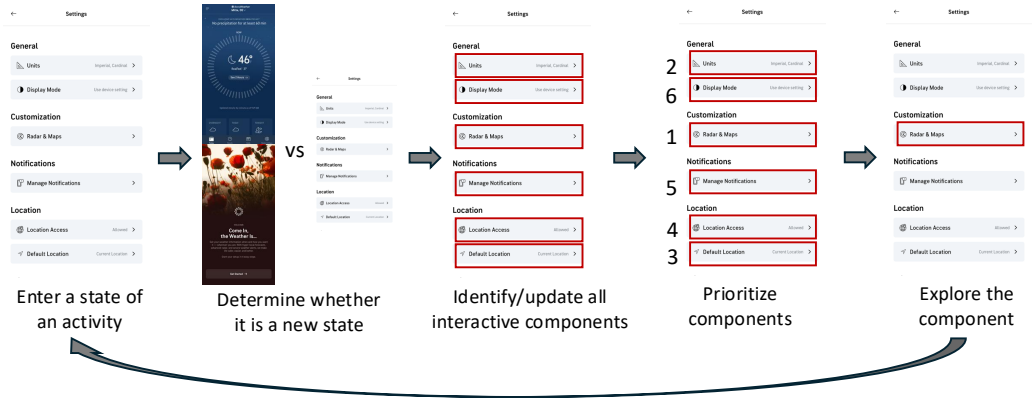


Fig. 8. Steps to explore an activity

Table 2. Distribution of expected actions of the missed transitions

Actions	Proportion
Missing pre-step (e.g., log in)	44.1%
ImageButton → TextView	28.5%
back	10.5%
TextView	6.2%
ImageButton	5.2%
Complex logic (more than three actions)	4.9%
TextView → ImageButton	0.3%
TextView → TextView	0.2%
ImageButton → TextView → ImageButton	0.2%

taken, leading to a series of missed transitions. For example, if the tool cannot visit the first screen in Figure 8, it cannot explore the following screens. Therefore, visiting the first screen in Figure 8 is a key step in exploring the app. Transitions requiring taking multiple actions simultaneously is commonly difficult to achieve, which is the failure reason for over 34.1% (=28.5% + 4.9% + 0.3% + 0.2% + 0.2%) of missed transitions. Transitions requiring actions on the image buttons are more likely to be missed than transitions related to the text-related widgets. Beyond those common issues, various ATG generation tools face distinct challenges based on their underlying technology stack.

Vision-based tools could incorrectly consider the same state as different states due to subtle visual differences. Existing visual-based tools, e.g., Humanoid, usually first take a screenshot of the app and then use the visual information in the screenshots to visually determine the state of the app. Specifically, Humanoid considers a state as a two-channel UI skeleton image. The first channel of the image renders the bounding box regions of text UI elements, while the second channel renders the bounding box regions of non-text UI elements.

However, Figure 9 shows that such visual-based tools fail to identify the commonality of these two states, and they consider these two screens of the app as two different states. In fact, these two screens are the same in terms of functionality but are slightly different in terms of visual

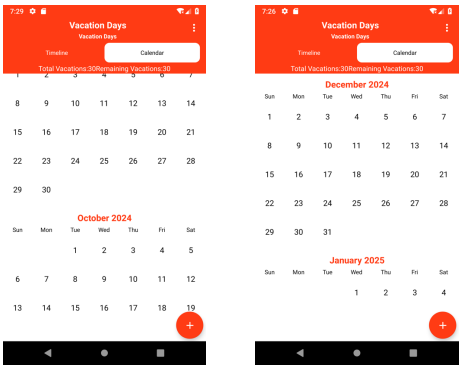


Fig. 9. Humanoid fails to distinguish different states visually

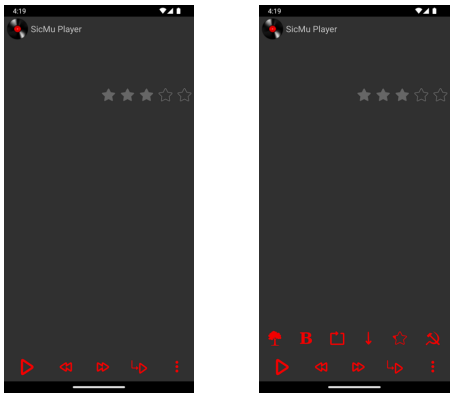
information. Wrongly identifying too many states in the app would result in the explosion of the states. The following-up exploration algorithm would explore more on the same activity, which makes the weight of the activity different from the designation, leading to the invalidation of the exploration strategy. We suggest that vision-based state identification algorithms also use other page attributes to reduce the number of identified states.

Learning-based tools may overlook subtle distinctions between states, potentially misclassifying different states as identical. For example, Q-Testing compares the widget composition to determine whether the app is in a new state. Specifically, it first uses UIAutomator to dump information about the widgets in the current screen; then it trains a Siamese LSTM network to extract the features of the widgets in the current screen and conduct the comparison by giving a probability of whether the states are the same. If the current state is similar to any prior state, it will not be considered a new state, and no more actions related to interacting with the app will be added (Section 3.1 [30]).

However, Figure 10 shows that after a user clicks the `More` button on the screen, Q-Testing still considers the newer screen as a visited state, and no more actions are added. This indicates that Q-Testing fails to distinguish different states. This makes Q-Testing still interact with the actions identified before clicking the button, which overlooks the new buttons presented on the new screen. Even worse, more functionalities of this app are hidden in the buttons that appear after the click. This means few activities explored for this app when using Q-Testing. One possible reason is that the widgets' attributes provide limited information to the model, leading the model to consider it less likely to be in a different state.

Widget-based tools might fail to capture the semantic knowledge in textual information. For example, Stoat uses the widget composition to determine whether the app is in a new state. To avoid the explosion of states, Stoat considers two widgets to be the same without considering the text information in the list. However, in some apps, different textual information indicates different types of resources, while different types of resources require different types of file-handler and will be redirected to different activities. For example, in Figure 11, the `.pdf` file will be redirected to the `amazeutilsalias` activity , while `.csv` file will be redirected to the `TextEditor` activity. Ignoring textual information can cause the ATG generation tool to overlook actionable GUI widget compositions, leading to missed activity transitions.

Learning-based tools also could only explore the widgets that are more similar to be explored in their training data. Humanoid is the learning-based tool that explores the app by



(a) Different states with different widgets.

```
1  ========episode 4_1========
2  the state has been visited before
3  the selected actiStopping: souch.smp
4  Starting: Intent { cmp=souch.smp/.Main }
5  on string is     11@click(resource-id='souch.smp:id/more_button', instance='0')
       :android.widget.ImageButton@""
6  info is   mCurrentFocus=Window{438b9f5 u0 souch.smp/souch.smp.Main}
7  current_activity_name is: souch.smp.Main
8  current existing state id :1
9  the updated action is     11@click(resource-id='souch.smp:id/more_button',
       instance='0'):android.widget.ImageButton@""
10
11 the jumped state is souch.smp.Main
12
13 ========episode 4_2========
14 the state has been visited before
15 ...
```

(b) Corresponding log.

Fig. 10. Q-Testing fails to distinguish different states because it fails to compare the attributes of the widgets

learning from how users use it. Specifically, they train a model that uses convolutional layers to capture the GUI visual information and residual LSTM modules to capture the interaction context information. Finally, they used deconvolutional layers and fully connected layers to generate the distribution of the location and type of the action, respectively. Specifically, to train the model, they extracted interaction flows from the Rico dataset [11], where each interaction flow contained 24.8 actions on average.

However, only the widgets commonly used by users in the Rico dataset [11] will be explored by Humanoid. If there is a widget that is not similar to the widgets in the Rico dataset [11], the widget is less likely to be explored. For example, Figure 12 shows that Humanoid fails to explore the app by clicking the About button. One possible reason is that few apps use a skull as the icon

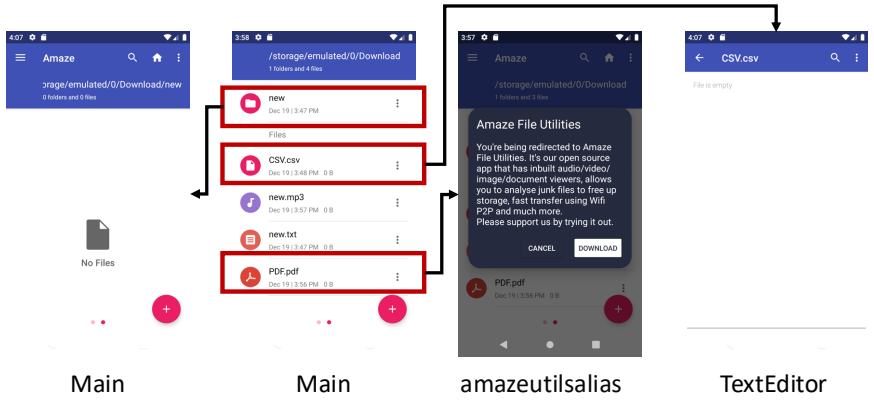


Fig. 11. STOAT overlooks the semantic knowledge in textual information

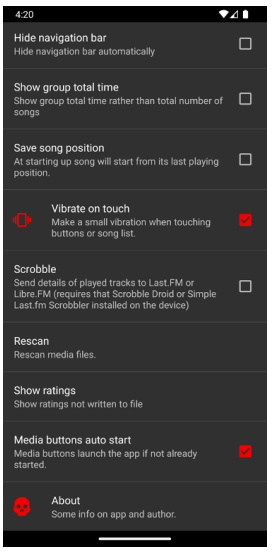


Fig. 12. HUMANOID fails to distinguish different states visually

of `About` button, and this rarely appears in the Rico dataset [11]. Therefore, HUMANOID trained using the Rico dataset [11] will not likely explore such buttons.

Rule-based tools could struggle to adhere to specific consecutive execution sequences required to trigger certain widget actions. For example, when assigning different widgets with different priorities, STOAT considers the event type, the number of unvisited children of the widget, and the history execution frequencies. Intuitively, if a widget is explored, its priority should be degraded in the future exploration process. Therefore, the widgets frequently executed in the past are assigned with a lower weight.

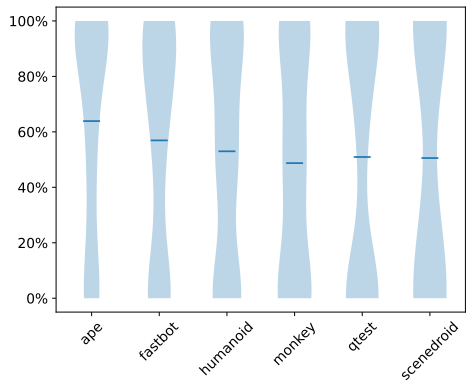
However, some hidden widgets are only available when a certain widget is explored. For example, in Figure 10, when the `More` button is clicked, the sub-menu is shown. And the transition to another activity, i.e., visiting `Settings` activity by clicking the `Setting` button, is only available in the

sub-menu. However, when using Stoat, the frequent clicking of the `More` button makes the priority of the setting button always lower. This makes the Stoat fail further to click the `Setting` button and fail to visit the `Settings` activity.

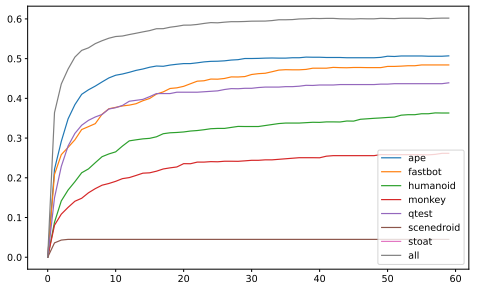
> We identify that ATG generation tools would commonly fail when (1) determining whether the app is in a new state and (2) prioritizing the components to explore. Such failure would cause the exploration to be trapped in a loop.

5 DISCUSSION

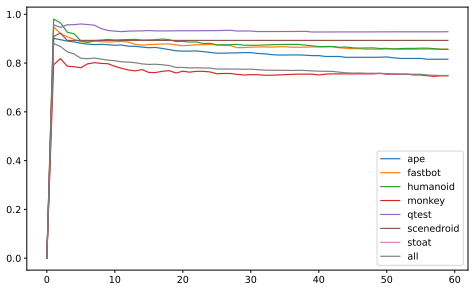
5.1 Can we run the tools for a shorter time and then combine the results of different tools as the final result?



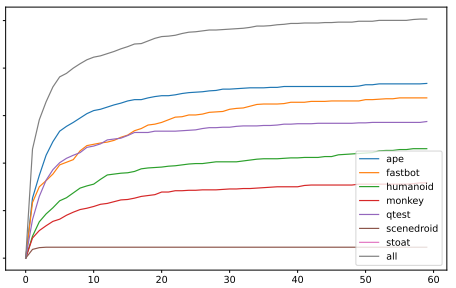
(a) Proportion of transitions identified in the Logcat file among all transitions identified by tools. The bars are the mean values.



(b) Overall F1-scores of different tools along time.



(c) Overall precisions of different tools along time.



(d) Overall recalls of different tools along time.

Fig. 13. Performance of different tools along the time.

Section 4.1.1 shows that different tools report different sets of transitions because of the different capabilities of different tools. Though the combined results can introduce incorrect transitions falsely reported by different tools, the combined results can complete tools from each other to identify more correct transitions. However, running all the tools for a long time and then combining their results would result in a low return on investment. We want to understand how long it takes to run each tool to get a satisfactory result.

We use the same approach as Section 4.1.2 to extract the time when a transition is conducted. Figure 13a shows the proportion of transitions recorded in Logcat. We observe that not all transitions reported by the tools are recorded in Logcat. This is because, as indicated in Section 4.2, many activities are explored before they are fully rendered. Thus, there is no `Display` related log of such activities. Considering that we only discuss the performance of different tools using the same setting here, we believe our comparison is fair.

Figure 13 shows the performance of different tools along the execution time. We observe that the performances of different tools are stable over time, and the conclusions in Section 4.1 are also valid along the execution process of different tools over time. For example, APE is better than all other tools along the time.

We notice that the recall of the combined results of different tools with the first 10-minute run is 0.43, which is 15.1% better than the results of the best ATG generation tool, i.e., APE, after a 60-minute run. **We recommend that to identify more real transitions, instead of running a single tool for a long time, users could run all the tools together for a shorter time (e.g., 10 minutes) and then combine the results from the tools.**

5.2 Implications

Dynamic testing tools should adopt longer action intervals during UI page transition. Specifically, dynamic testing tools should wait for the pages to be fully rendered. Section 4.2 shows that the ATG generation tools can report incorrect transitions because the tool can take action before the page is fully rendered. The tool can explore the widgets in the new activity while it detects that the app is still in the last activity or explore the widgets in the last activity while it detects that the app is in the new activity. We suggest that future researchers also take the time to fully render pages into consideration when designing dynamic testing tools [2, 14].

Future researchers should carefully parse the results of ATG generation tools. Section 4.2 shows that the ATG generation tools can report incorrect transitions because the results of the tools can also record `Start` of an activity rather than the `Display` of an activity. However, there can be activities that behave like routers, redirecting the app to another activity but never displaying it on the screen. When we aim to explore the functionalities of an app and model the app from the frontend UI, we do not need such transitions. Though using Logcat to filter out the activities that are only started but not displayed seems to be viable, there also can be activities explored before fully rendered (i.e., the activities are only `Started` in the Logcat, without the log related to `Display`). This indicates that using Logcat to filter out such transitions is not feasible. Future researchers could design approaches to remove such transitions in the ATG.

Future researchers could use static analysis techniques that are fast, accurate, and more compatible with Android API versions. Section 4.3 shows that SceneDroid frequently crashes due to the sensitive static analysis. However, we observe that many transitions can only be identified by SceneDroid. This indicates that though SceneDroid can be problematic sometimes, we still can gain insights from SceneDroid. For example, we acknowledge the value of the static information extracted from static analysis. However, to use such knowledge, we suggest that future researchers use static analysis with fast techniques that are not sensitive to the API version. For example, future researchers could leverage LLM, which enriches the knowledge of program analysis, to conduct static analysis.

When training models to guide tools to explore applications, more high-quality datasets are needed. Section 4.3 shows that Humanoid can fail to explore widgets that are dissimilar to the explored widgets in the training data. Considering that a large number of tools are now based on machine learning, more high-quality data is needed. Future work can consider contributing such high-quality datasets.

Researchers could take multiple information, e.g., visual, text, and attributes of widgets, into consideration to determine the states in an app and prioritize the widgets to explore. In Section 4.3, we observe that visual-based tools, e.g., Humanoid, fail to determine the state of an app and can often introduce too many states into the model. At the same time, the widget-based tools, e.g., Stoat, may ignore the semantic knowledge in textual information and miss the possible transitions; probability-based tools may ignore changes in the limited attribute information of widgets and incorrectly different states as the same state. Moreover, when prioritizing the widgets to explore, these tools only consider a single type of information and overlook others. Considering that LLMs have proven their capabilities in many fields, future work can consider combining different types of information (e.g., widget attributes, visual information, and semantic information of text) with the help of LLM to distinguish states and prioritize the widgets to explore.

5.3 Limitations and Threats to validity

Threats to external validity relate to the generalizability of our findings. To understand the performance of different tools in generating ATG transitions, we conduct experiments with different tools using 98 apps released between October 2022 and October 2024. These apps must have a minimum SDK version lower than 28, include more than two activities declared in the manifest file, and utilize a common Application Binary Interface (ABI) that can run on an emulator hosted on an x86_64 Linux server. We acknowledge that the generalizability of our results is limited by the specific technical setup and the selection of apps used in our study. However, it takes 2,100+ hours in total to conduct the experiment, indicating the analysis is expensive. Considering 98 apps are of a similar scale with other studies [32, 34], and we tried our best to collect apps from multiple sources (i.e., 53 apps are collected from Google Play via AndroZoo, and 45 apps are collected from F-Droid.) that have been released recently, we believe our collected apps are representative. We encourage future research to explore the performance of ATG generation tools on a broader range of apps and configurations. This will help to validate our findings and assess the generalizability of the results across different scenarios. We believe that our study provides a valuable foundation for future research in this area, and we hope that our findings will inspire further investigations into the performance of ATG generation tools in various contexts.

Moreover, the criteria we use to compare the tools are specific to our research questions. Different tools may try to solve different problems, and maybe they perform better on the problem they try to solve. For example, if the target is to produce as many crashes as possible or find the activities that are more important to end-users, the target may not necessarily be to find all activities.

Threats to internal validity relate to experiment bias and errors. We heavily depend on manual processes to obtain the ground truth ATGs. Like any human activity, our manual classification can be subject to personal bias and subjectivity. To reduce personal bias in the manual labeling process, as indicated in Section 3.3, each app was labeled by two of the authors using the results of the ATG generation tools as a supplementary reference. Few disagreements were reported and were discussed by the two authors until a consensus was reached. This gives us high confidence in the dataset used in our paper.

We acknowledge that the Android version used in our study is not the latest versions available. This is because we want to ensure that the Android version used in our study is compatible with the ATG generation tools we evaluate. Additionally, this version allows us to assess the performance of these tools on a range of apps. However, we believe that the insights gained from our study still provide valuable contributions to the field, especially in understanding the challenges and limitations of current ATG generation tools. Our findings can serve as a foundation for future research that explores the performance of ATG generation tools on more recent Android versions. We encourage future researchers to build upon our work by evaluating ATG generation tools on

the latest Android versions to assess their performance and identify any new challenges that may arise. This will help ensure that the findings remain relevant and applicable to the current state of the Android ecosystem.

6 CONCLUSION AND FUTURE WORK

In this paper, we empirically explore the capability of ATG generation tools. We evaluate the performance of 7 popular tools that generate ATGs and corresponding screenshots of the activities so that users can easily understand app functionalities. We manually identify all transitions in 89 real-world apps and compare the results reported by these tools. We observe that all tools can miss transitions and report incorrect transitions at the same time. APE is the best tool that can identify the most number of transitions and report the least number of incorrect transitions. Different tools do not identify the same set of transitions. APE is one of the fastest tools, and Humanoid is one of the slowest tools. We summarize five potential limitations of current tools that can cause dynamic testing tools to fail to identify transitions in ATG. Based on our findings, we propose a series of suggestions for users and researchers. Using the paper's findings, we plan to design a more robust tool to identify more transitions in ATG in the future. For example, we plan to combine different types of information (e.g., widget attributes, visual information, and text semantic information) with the help of LLM to distinguish states and prioritize the widgets to explore.

ACKNOWLEDGEMENT

This research / project is supported by the National Research Foundation, Singapore, and Cyber Security Agency of Singapore under its National Cybersecurity Research and Development Programme, NCRP25-P03-NCR-TAU. Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of National Research Foundation, Singapore and Cyber Security Agency of Singapore. This work is also supported by the Blavatnik Interdisciplinary Cyber Research Center at Tel Aviv University as part of a collaboration with CSA Singapore.

REFERENCES

[1] [n. d.]. F-Droid - Free and Open Source Android App Repository — f-droid.org. https://f-droid.org/. [Accessed 21-01-2025].

[2] [n. d.]. how to set idle timeout period? · Issue #116 · openatx/uiautomator2. https://github.com/openatx/uiautomator2/issues/116

[3] Faridah Akinotcho, Lili Wei, and Julia Rubin. 2025. Mobile Application Coverage: The 30% Curse and Ways Forward. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, Ottawa, ON, Canada, 2751–2763. https://doi.org/10.1109/icse55347.2025.00142

[4] Kevin Allix, Tegawendé F. Bissyandé, Jacques Klein, and Yves Le Traon. 2016. AndroZoo: collecting millions of Android apps for the research community. In *Proceedings of the 13th International Conference on Mining Software Repositories*. ACM, Austin Texas, 468–471. https://doi.org/10.1145/2901739.2903508

[5] Tanzirul Azim and Iulian Neamtiu. 2013. Targeted and depth-first exploration for systematic testing of android apps. In *Proceedings of the 2013 ACM SIGPLAN international conference on Object oriented programming systems languages & applications*. ACM, Indianapolis Indiana USA, 641–660. https://doi.org/10.1145/2509136.2509549

[6] Farnaz Behrang, Steven P. Reiss, and Alessandro Orso. 2018. GUIfetch: supporting app design and development through GUI search. In *Proceedings of the 5th International Conference on Mobile Software Engineering and Systems*. ACM, Gothenburg Sweden, 236–246. https://doi.org/10.1145/3197231.3197244

[7] Amiangshu Bosu, Fang Liu, Danfeng (Daphne) Yao, and Gang Wang. 2017. Collusive Data Leak and More: Large-scale Threat Analysis of Inter-app Communications. In *Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security*. ACM, Abu Dhabi United Arab Emirates, 71–85. https://doi.org/10.1145/3052973.3053004

[8] Haonan Chen, Daihang Chen, Yonghui Liu, Xiaoyu Sun, and Li Li. 2024. Are Your Android App Analyzers Still Relevant?. In *Proceedings of the IEEE/ACM 11th International Conference on Mobile Software Engineering and Systems*. 69–73.

[9] Sen Chen, Lingling Fan, Chunyang Chen, and Yang Liu. 2023. Automatically Distilling Storyboard With Rich Features for Android Apps. *IEEE Transactions on Software Engineering* 49, 2 (Feb. 2023), 667–683. https://doi.org/10.1109/TSE.2022.3159548

[10] Sen Chen, Lingling Fan, Chunyang Chen, Ting Su, Wenhe Li, Yang Liu, and Lihua Xu. 2019. StoryDroid: Automated Generation of Storyboard for Android Apps. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, Montreal, QC, Canada, 596–607. https://doi.org/10.1109/ICSE.2019.00070

[11] Biplab Deka, Zifeng Huang, Chad Franzen, Joshua Hibschman, Daniel Afergan, Yang Li, Jeffrey Nichols, and Ranjitha Kumar. 2017. Rico: A Mobile App Dataset for Building Data-Driven Design Applications. In *Proceedings of the 30th Annual ACM Symposium on User Interface Software and Technology (UIST '17)*. Association for Computing Machinery, New York, NY, USA, 845–854. https://doi.org/10.1145/3126594.3126651

[12] Mattia Fazzini, Martin Prammer, Marcelo d'Amorim, and Alessandro Orso. 2018. Automatically translating bug reports into test cases for mobile apps. In *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*. ACM, Amsterdam Netherlands, 141–152. https://doi.org/10.1145/3213846.3213869

[13] Sidong Feng and Chunyang Chen. 2022. GIFdroid: automated replay of visual bug reports for Android apps. In *Proceedings of the 44th International Conference on Software Engineering*. ACM, Pittsburgh Pennsylvania, 1045–1057. https://doi.org/10.1145/3510003.3510048

[14] Sidong Feng, Mulong Xie, and Chunyang Chen. 2023. Efficiency Matters: Speeding Up Automated Testing with GUI Rendering Inference. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, Melbourne, Australia, 906–918. https://doi.org/10.1109/ICSE48619.2023.00084

[15] Android Google. 2024. UI/Application Exerciser Monkey | Android Studio. https://developer.android.com/studio/test/other-testing-tools/monkey

[16] Tianxiao Gu, Chengnian Sun, Xiaoxing Ma, Chun Cao, Chang Xu, Yuan Yao, Qirun Zhang, Jian Lu, and Zhendong Su. 2019. Practical GUI Testing of Android Applications Via Model Abstraction and Refinement. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, Montreal, QC, Canada, 269–280. https://doi.org/10.1109/ICSE.2019.00042

[17] Han Hu, Han Wang, Ruiqi Dong, Xiao Chen, and Chunyang Chen. 2024. Enhancing GUI exploration coverage of Android apps with deep link-integrated Monkey. *ACM Transactions on Software Engineering and Methodology* 33, 6 (2024), 1–31.

[18] Duling Lai and Julia Rubin. 2019. Goal-Driven Exploration for Android Applications. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, San Diego, CA, USA, 115–127. https://doi.org/10.1109/ASE.2019.00021

[19] Yuanchun Li, Ziyue Yang, Yao Guo, and Xiangqun Chen. 2019. Humanoid: A Deep Learning-Based Approach to Automated Black-box Android App Testing. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, San Diego, CA, USA, 1070–1073. https://doi.org/10.1109/ASE.2019.00104

[20] Changlin Liu, Hanlin Wang, Tianming Liu, Diandian Gu, Yun Ma, Haoyu Wang, and Xusheng Xiao. 2022. ProMal: precise window transition graphs for android via synergy of program analysis and machine learning. In *Proceedings of the 44th International Conference on Software Engineering*. ACM, Pittsburgh Pennsylvania, 1755–1767. https://doi.org/10.1145/3510003.3510037

[21] Jiakun Liu, Zicheng Zhang, Xing Hu, Ferdian Thung, Shahar Maoz, Debin Gao, Eran Toch, Zhipeng Zhao, and David Lo. 2024. MiniMon: Minimizing Android Applications with Intelligent Monitoring-Based Debloating. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. ACM, Lisbon Portugal, 1–13. https://doi.org/10.1145/3597503.3639113

[22] Yonghui Liu, Xiao Chen, Pei Liu, John Grundy, Chunyang Chen, and Li Li. 2023. ReuNify: A Step Towards Whole Program Analysis for React Native Android Apps. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 1390–1402. https://doi.org/10.1109/ASE56229.2023.00113

[23] Yonghui Liu, Xiao Chen, Pei Liu, Jordan Samhi, John Grundy, Chunyang Chen, and Li Li. 2024. Demystifying React Native Android Apps for Static Analysis. *ACM Trans. Softw. Eng. Methodol.* (Nov. 2024). https://doi.org/10.1145/3702977 Just Accepted.

[24] Zhe Liu, Chunyang Chen, Junjie Wang, Yuekai Huang, Jun Hu, and Qing Wang. 2022. Guided Bug Crush: Assist Manual GUI Testing of Android Apps via Hint Moves. In *CHI Conference on Human Factors in Computing Systems*. ACM, New Orleans LA USA, 1–14. https://doi.org/10.1145/3491102.3501903

[25] Zhe Liu, Chunyang Chen, Junjie Wang, Yuhui Su, Yuekai Huang, Jun Hu, and Qing Wang. 2023. Ex pede Herculem: Augmenting Activity Transition Graph for Apps via Graph Convolution Network. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, Melbourne, Australia, 1983–1995. https://doi.org/10.1109/ICSE48619.2023.00168

[26] Zhengwei Lv, Chao Peng, Zhao Zhang, Ting Su, Kai Liu, and Ping Yang. 2022. Fastbot2: Reusable Automated Model-based GUI Testing for Android Enhanced by Reinforcement Learning. In *Proceedings of the 37th IEEE/ACM International*

Conference on Automated Software Engineering. ACM, Rochester MI USA, 1–5. https://doi.org/10.1145/3551349.3559505

[27] Kevin Moran, Carlos Bernal-Cardenas, Michael Curcio, Richard Bonett, and Denys Poshyvanyk. 2020. Machine Learning-Based Prototyping of Graphical User Interfaces for Mobile Apps. *IEEE Transactions on Software Engineering* 46, 2 (Feb. 2020), 196–221. https://doi.org/10.1109/TSE.2018.2844788

[28] Damien Octeau, Daniel Luchaup, Matthew Dering, Somesh Jha, and Patrick McDaniel. 2015. Composite Constant Propagation: Application to Android Inter-Component Communication Analysis. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*. IEEE, Florence, Italy, 77–88. https://doi.org/10.1109/ICSE.2015.30

[29] Damien Octeau, Patrick McDaniel, Somesh Jha, Alexandre Bartel, Eric Bodden, Jacques Klein, and Yves Le Traon. 2013. Effective inter-component communication mapping in Android with Epicc: an essential step towards holistic security analysis. In *Proceedings of the 22nd USENIX conference on Security (SEC'13)*. USENIX Association, USA, 543–558.

[30] Minxue Pan, An Huang, Guoxin Wang, Tian Zhang, and Xuandong Li. 2020. Reinforcement learning based curiosity-driven testing of Android applications. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*. ACM, Virtual Event USA, 153–164. https://doi.org/10.1145/3395363.3397354

[31] Ting Su, Guozhu Meng, Yuting Chen, Ke Wu, Weiming Yang, Yao Yao, Geguang Pu, Yang Liu, and Zhendong Su. 2017. Guided, stochastic model-based GUI testing of Android apps. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE 2017)*. Association for Computing Machinery, New York, NY, USA, 245–256. https://doi.org/10.1145/3106237.3106298

[32] Ting Su, Jue Wang, and Zhendong Su. 2021. Benchmarking automated GUI testing for Android against real-world bugs. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, Athens Greece, 119–130. https://doi.org/10.1145/3468264.3468620

[33] Mian Wan, Yuchen Jin, Ding Li, Jiaping Gui, Sonal Mahajan, and William G. J. Halfond. 2017. Detecting display energy hotspots in Android apps. *Software Testing, Verification and Reliability* 27, 6 (Sept. 2017), e1635. https://doi.org/10.1002/stvr.1635

[34] Wenyu Wang, Dengfeng Li, Wei Yang, Yurui Cao, Zhenwen Zhang, Yuetang Deng, and Tao Xie. 2018. An empirical study of Android test generation tools in industrial cases. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*. ACM, Montpellier France, 738–748. https://doi.org/10.1145/3238147.3240465

[35] Xusheng Xiao, Xiaoyin Wang, Zhihao Cao, Hanlin Wang, and Peng Gao. 2019. IconIntent: Automatic Identification of Sensitive UI Widgets Based on Icon Classification for Android Apps. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, Montreal, QC, Canada, 257–268. https://doi.org/10.1109/ICSE.2019.00041

[36] Jiwei Yan, Shixin Zhang, Yepang Liu, Xi Deng, Jun Yan, and Jian Zhang. 2022. A Comprehensive Evaluation of Android ICC Resolution Techniques. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. ACM, Rochester MI USA, 1–13. https://doi.org/10.1145/3551349.3560420

[37] Jiwei Yan, Shixin Zhang, Yepang Liu, Jun Yan, and Jian Zhang. 2022. ICCBot: Fragment-Aware and Context-Sensitive ICC Resolution for Android Applications. In *2022 IEEE/ACM 44th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE, Pittsburgh, PA, USA, 105–109. https://doi.org/10.1109/ICSE-Companion55297.2022.9793791

[38] Shengqian Yang, Hailong Zhang, Haowei Wu, Yan Wang, Dacong Yan, and Atanas Rountev. 2015. Static Window Transition Graphs for Android (T). In *2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, Lincoln, NE, USA, 658–668. https://doi.org/10.1109/ASE.2015.76

[39] Hailong Zhang, Haowei Wu, and Atanas Rountev. 2016. Automated test generation for detection of leaks in Android applications. In *Proceedings of the 11th International Workshop on Automation of Software Test*. ACM, Austin Texas, 64–70. https://doi.org/10.1145/2896921.2896932

[40] Xiangyu Zhang, Lingling Fan, Sen Chen, Yucheng Su, and Boyuan Li. 2023. Scene-Driven Exploration and GUI Modeling for Android Apps. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, Luxembourg, Luxembourg, 1251–1262. https://doi.org/10.1109/ASE56229.2023.00179

[41] Yixue Zhao, Marcelo Schmitt Laser, Yingjun Lyu, and Nenad Medvidovic. 2018. Leveraging program analysis to reduce user-perceived latency in mobile applications. In *Proceedings of the 40th International Conference on Software Engineering*. ACM, Gothenburg Sweden, 176–186. https://doi.org/10.1145/3180155.3180249